Prerequisites — Setting Up Your Kubernetes Cluster on GCP

Before the first workshop, you need a running Kubernetes cluster on Google Cloud Platform. Follow every step in order. If something fails, stop and ask for help.

Estimated time: **30–45 minutes**

Step 1 — Create a Google Cloud Account

Google Cloud offers a **free trial with $300 in credits** valid for 90 days.

> **Before you start:** The free trial includes $300 in credits valid for 90 days — no charges are made during the trial period. If you are unsure whether your account is eligible, check console.cloud.google.com/billing or ask your instructor.

1. Go to cloud.google.com and click **Get started for free**.
2. Sign in with your Google account.
3. Follow the signup form. You will be asked to enter a **credit card** — this is required by Google to verify your identity.
4. After activation you will land in the **Google Cloud Console** at console.cloud.google.com.

Step 2 — Note Your Project ID

Google Cloud automatically creates a default project for you.

1. In the Cloud Console, look at the top bar — you will see your project name next to the Google Cloud logo.
2. Click on the project name to open the project selector.
3. Copy the **Project ID** — it looks like: `my-project-123456`

> The **Project ID** is different from the **Project Name**. You need the ID (all lowercase, may include numbers).

You will need this in Step 6.

Step 3 — Open Google Cloud Shell

Cloud Shell is a browser-based terminal with all required tools pre-installed (gcloud, kubectl, terraform). No local installation is needed.

1. In the Cloud Console, click the **terminal icon** (`>_`) in the top-right toolbar.
2. A terminal panel will open at the bottom of the page.
3. Wait a few seconds for the shell to start.

> All remaining steps are run inside Cloud Shell.

Step 4 — Generate an SSH Key Pair

The cluster VM needs an SSH key for configuration.

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -N ""
```

Verify the keys were created:

```
ls ~/.ssh/id_rsa ~/.ssh/id_rsa.pub
```

Step 5 — Download the Workshop Scripts

```
cd ~
git clone https://github.com/matysiaq/prin-2026-observability.git
cd prin-2026-observability/scripts/gcp-scripts
```

Step 6 — Configure the Cluster

Open the configuration file in an editor of your choice:

```
vim config.yaml      # press i to enter insert mode, Esc then :wq to save and exit
nano config.yaml     # Ctrl+X, then Y, then Enter to save and exit
```

Find the line that says:

```
project: "TODO: your-project-id"
```

Replace it with your Project ID from Step 2. For example:

```
project: "my-project-123456"
```

You do not need to change anything else.

Step 7 — Deploy the Cluster

Make the script executable and run it:

```
chmod +x deploy.sh
./deploy.sh
```

The script will:

1. Install Ansible automatically (not pre-installed in Cloud Shell — takes ~1 minute)
2. Enable the Compute Engine API for your project
3. Create 1 VM on GCP via Terraform (~3 minutes)
4. Configure Kubernetes on the VM via Ansible (~10 minutes)

Total wait time: approximately **15 minutes**.

You will see progress messages starting with `==>`. Do not close Cloud Shell during this time.

Step 8 — Verify the Cluster

When the script finishes, you will see:

```
============================================================
  Cluster 'k8s-workshop' is ready!
  Mode:       3-node (1 master + 2 workers, e2-standard-2)
  Kubeconfig: /home/.../kubeconfigs/k8s-workshop.config
============================================================
```

Point `kubectl` at the new cluster:

```
export KUBECONFIG=~/prin-2026-observability/scripts/gcp-scripts/ansible/kubeconfigs/k8s-workshop.config
```

Check that the node is `Ready`:

```
kubectl get nodes
```

Expected output (may take up to 2 minutes — workers may briefly show `NotReady` while Cilium CNI initializes):

```
NAME                   STATUS   ROLES           AGE   VERSION
k8s-workshop-master    Ready    control-plane   5m    v1.30.0
k8s-workshop-worker-1  Ready    <none>          4m    v1.30.0
k8s-workshop-worker-2  Ready    <none>          4m    v1.30.0
```

Step 9 — Add KUBECONFIG to Your Shell Profile (recommended)

So you don't have to re-export KUBECONFIG each time you open Cloud Shell:

```
echo 'export KUBECONFIG=~/prin-2026-observability/scripts/gcp-scripts/ansible/
kubeconfigs/k8s-workshop.config' >> ~/.bashrc
source ~/.bashrc
```

Step 10 — Copy Kubeconfig to Your Local Machine (optional)

If you want to use `kubectl` from your own laptop instead of Cloud Shell, you need to copy the kubeconfig file locally. Your local machine must have `kubectl` installed.

In Cloud Shell, print the kubeconfig content:

```
cat ~/prin-2026-observability/scripts/gcp-scripts/ansible/kubeconfigs/k8s-
workshop.config
```

On your local machine, create the file and paste the content:

```
# Linux / macOS
mkdir -p ~/.kube
nano ~/.kube/k8s-workshop.config   # paste the content, save
export KUBECONFIG=~/.kube/k8s-workshop.config
```

```
# Windows (PowerShell)
New-Item -ItemType Directory -Force "$env:USERPROFILE\.kube"
notepad "$env:USERPROFILE\.kube\k8s-workshop.config"   # paste the content, save
$env:KUBECONFIG = "$env:USERPROFILE\.kube\k8s-workshop.config"
```

Verify it works:

```
kubectl get nodes
```

> The kubeconfig already points to the cluster's external IP, so it works from any network without modification.

Cost Reminder

Your cluster costs approximately **$0.20/hour** while running (3 × e2-standard-2 in europe-central2). When you are not working, stop all VMs to save credits:

```
# Stop all nodes (static IPs are preserved)
gcloud compute instances stop k8s-workshop-master k8s-workshop-worker-1 k8s-workshop-worker-2 --zone europe-central2-a

# Start them again
gcloud compute instances start k8s-workshop-master k8s-workshop-worker-1 k8s-workshop-worker-2 --zone europe-central2-a
```

> The static IPs **do not change** when you stop and start the VMs. Your `kubectl` will keep working after a restart.

> **Note:** After starting the VMs, wait about 2 minutes for all nodes to become `Ready` before using the cluster.

Cleanup After the Workshop

To delete all resources and stop any charges:

```
cd ~/prin-2026-observability/scripts/gcp-scripts/terraform
terraform destroy -auto-approve
```

Troubleshooting

Problem	Solution
`ERROR: SSH public key not found`	Run Step 4 again
`ERROR: Please set your GCP project ID`	Edit config.yaml and fill in `gcp.project`
`kubectl: connection refused`	Wait 2 more minutes — API server may still be starting
Node stuck in `NotReady`	Run `kubectl describe node <name>` and show output to instructor
Cloud Shell disconnected mid-deploy	Re-open Cloud Shell and run `./deploy.sh --skip-terraform`
GCP asks for credit card again	Your trial may not have activated — check console.cloud.google.com/billing
`PERMISSION_DENIED: Compute Engine API not enabled`	Run: `gcloud services enable compute.googleapis.com --project=YOUR_PROJECT_ID`